

GraphSAGE Model Implementation for Illicit Transaction Detection on the Bitcoin Elliptic Dataset

Andrea Colombo - GPU Computing final course project - A.A 2025/2026
Università degli studi di Milano

Abstract — In this report we present a graph-based model trained to identify illicit transactions on the Bitcoin blockchain.

We describe the task, the model definition and the reasoning behind its architecture, performance considerations, and an evaluation of the speedup provided by GPU execution relative to CPU execution.

All the relevant code is presented in the Jupyter Notebook provided alongside this report.

Table of Contents

1	Introduction	2
1.1	Motivation and Objectives	2
1.2	Development Environment	2
2	Dataset Description	3
2.1	Related Challenges	3
3	Model Definition	4
4	Training Methodology	5
4.1	Handling Class Imbalance	5
4.2	Optimizer and Hyperparameters	5
4.3	Early Stopping	6
4.4	Threshold Calibration	6
4.5	Preventing Test-Set Leakage	6
5	Experimental Results	6
5.1	Test Set Performance	6
5.2	GPU Speedup	7
6	Conclusion	9
	Bibliography	9

1 Introduction

Anti-money laundering (AML) is becoming increasingly relevant in cryptocurrency networks. The decentralized nature of the blockchain makes it an attractive environment for illicit actors, including scammers, terrorism financiers, and operators of Ponzi schemes.

The permanent nature of the blockchain, where every transaction is stored on a public ledger, enables large scale forensic analysis that is difficult to replicate on other financial systems.

The flow of funds inside the blockchain can be described as a directed graph, making it a great target for graph based deep learning approaches.

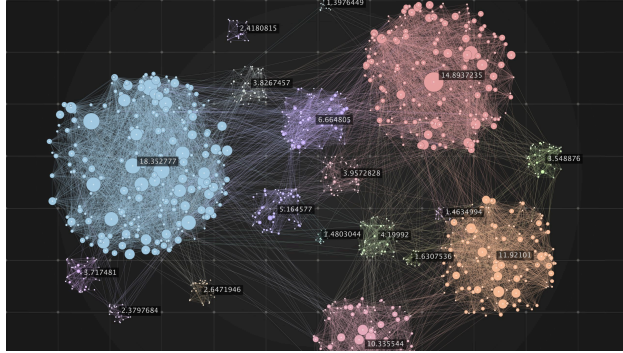


Figure 1: Visualization of a graph neural network’s structure.

1.1 Motivation and Objectives

A purely unsupervised pretraining objective, such as training a graph encoder to reconstruct the adjacency structure with a variational graph auto-encoder [1], built on the variational framework of [2], and only fitting a classifier afterwards on the frozen embeddings, provides no explicit incentive to preserve the specific signal that separates licit from illicit transactions.

Weber et al. reach a related conclusion in their own work on this dataset [3]: a Random Forest trained directly on raw transaction features outperforms a plain Graph Convolutional Network [4], suggesting that the local features carry most of the useful signal and that naive neighbourhood aggregation mostly dilutes it. This motivates training the encoder directly on the classification objective and providing the classifier with a direct path to the raw features, rather than relying on this information surviving the unsupervised aggregation process.

1.2 Development Environment

This project was developed using the computing resources provided by **Google Colab**; the notebook accompanying this report therefore contains environment-specific configurations that must be modified to run the code in a local environment.

The assigned runtime provided:

- a **Tesla T4** GPU (15.6 GB, compute capability 7.5)
- a 2-core **Intel Xeon** CPU @ 2.00GHz

The relevant software versions are the following:

- Python 3.13.15
- PyTorch 2.11.0+cu128
- PyTorch Geometric 2.8.0.post1.

2 Dataset Description

The Bitcoin Elliptic dataset represents Bitcoin transactions as a temporal graph: each node is a transaction, and each directed edge represents the flow of funds from one transaction to another. The graph contains 203769 nodes and 234355 edges. Weber et al. [3] describe 166 features per node: 94 “local” transaction features, including the time step, and 72 aggregated features computed over each transaction’s one-hop neighbourhood. The PyTorch Geometric loader used in this project excludes the time step from the feature tensor, recovering it separately to build the temporal train/test split (see Section 2.1); the model therefore receives 165 features per node (93 local and 72 aggregated).

2.1 Related Challenges

The Elliptic dataset presents several distinctive challenges that must be addressed in order to obtain meaningful results in our task:

- **The dataset is heavily imbalanced:** the vast majority of labelled transactions (about 90%) are licit. As a result, a model may report 90% accuracy while still failing to effectively detect illicit transactions. This characteristic required the use of additional metrics to evaluate model performance, including:
 - F1-score
 - Precision and recall
 - ROC-AUC, PR-AUC
 - Confusion matrix
- **Temporal nature of the graph:** transactions are grouped into 49 distinct timesteps, each roughly two weeks apart. The graph is also locally connected in time, as the majority of edges link transactions within the same or adjacent timesteps (as shown in [3]).

With a relatively small number of timesteps, a subset of them contain a disproportionate amount of illicit transactions compared to the others, for instance near a black market event. Consequently, a random train/test split does not fit this use case. Since the model must generalise well to future timesteps, the split instead trains the model on early timesteps and tests it on later ones.
- **Incomplete labels:** only about 23% of the graph’s nodes are labelled, leaving considerably less information to work with compared to other benchmark datasets.

3 Model Definition

GraphSAGE [5] is a graph neural network layer that, instead of operating on the full adjacency matrix like earlier spectral approaches, learns to produce a node’s embedding by aggregating the feature vectors of its local neighbourhood and combining the result with the node’s own representation. In its original formulation, this neighbourhood is randomly sampled to bound computational cost on large graphs; the implementation presented here instead aggregates over the full one-hop neighbourhood in a single full-batch forward pass, since the Elliptic graph’s size fits comfortably within the memory of a single GPU (see Section 5.2). Because the aggregation function is learned rather than tied to a specific fixed graph, the resulting model generalises to nodes and edges unseen during training, which makes it well suited to the Elliptic graph’s temporal train/test split (see Section 2.1).

The presented model consists of a two-layer **GraphSAGE encoder** that produces a 128-dimensional hidden representation, which is concatenated with the 165-dimensional input feature vector. This concatenation acts as a skip (residual) connection: an identity path that lets the input feature vector bypass the encoder and reach the classifier directly, side-stepping the vanishing-gradient and information-loss issues that can affect deep or heavily-transformed stacks of layers. The idea was popularised for deep convolutional networks by [6] and is used here to ensure the raw transaction features remain directly accessible to the classifier alongside the neighbourhood information learned by the encoder.

The encoder is followed by a two-layer MLP classifier.

The following code listing presents the model implementation using **PyTorch Geometric** [7]:

Python

```
class SkipGraphSAGE(torch.nn.Module):

    def __init__(self, in_chs, hidden_chs, dropout = 0.3):
        super(SkipGraphSAGE, self).__init__()
        self.sage_1 = SAGEConv(in_chs, hidden_chs)
        self.sage_2 = SAGEConv(hidden_chs, hidden_chs)
        self.bn_1 = torch.nn.BatchNorm1d(hidden_chs)
        self.bn_2 = torch.nn.BatchNorm1d(hidden_chs)
        self.dropout = dropout

        self.classifier = torch.nn.Sequential(
            torch.nn.Linear(in_chs + hidden_chs, hidden_chs),
            torch.nn.ReLU(),
            torch.nn.Dropout(dropout),
            torch.nn.Linear(hidden_chs, 1)
        )

    def embed(self, x, edge_index):
        h = self.sage_1(x, edge_index)
        h = self.bn_1(h).relu()
        h = torch.nn.functional.dropout(h, p = self.dropout, training = self.training)
        h = self.sage_2(h, edge_index)
        h = self.bn_2(h).relu()
        return h

    def forward(self, x, edge_index):
        h = self.embed(x, edge_index)
        z = torch.cat([x, h], dim = -1)
        return self.classifier(z).squeeze(-1)
```

Listing 1: Skip GraphSAGE model implementation

4 Training Methodology

As described in Section 2.1, this task requires specific engineering choices to ensure a fair evaluation and training process.

4.1 Handling Class Imbalance

The classifier was trained using a **weighted binary cross-entropy loss**:

$$\mathcal{L}_{\text{BCE}} = -w_p \cdot y \cdot \log(\sigma(z)) - (1 - y) \cdot \log(1 - \sigma(z))$$

where z is the classifier’s output logit, $y \in \{0, 1\}$ is the true label, σ is the sigmoid function, and w_p is the *positive class weight*, set to the licit-to-illicit ratio of the training partition: $w_p \approx 7.63$ (22,468 licit versus 2,943 illicit training examples). This penalizes a missed illicit transaction approximately 7.6 times more than a missed licit one, directly counteracting the imbalance in the loss function.

4.2 Optimizer and Hyperparameters

The model was trained using the **Adam** optimizer [8]. The following table summarises the hyperparameters used in the training pipeline.

Parameter	Value
hidden channels	128

dropout	0.3
max epochs	300
patience	20
learning rate	5e-3
weight decay	1e-4
validation fraction	0.15 (class-stratified)

4.3 Early Stopping

Given the class imbalance, PR-AUC on the validation set is a more informative stopping criterion than accuracy or raw loss [9]. Training runs for up to 300 epochs, evaluating validation PR-AUC every 5 epochs, retaining the best checkpoint, and stopping once performance has not improved for 20 epochs. This approach also mitigates overfitting, a common concern when training deep learning models on limited labelled data.

4.4 Threshold Calibration

Rather than using a fixed 0.5 cutoff, the decision threshold is computed after training by sweeping the validation **precision-recall curve** and selecting the value that maximises the illicit class’s F1-score.

4.5 Preventing Test-Set Leakage

Because GraphSAGE aggregates neighbour features, a naive implementation could allow information about test-partition nodes to reach the model before final evaluation, even without directly accessing their labels. If message passing at training or validation time runs over the full graph, embeddings for training and validation nodes are partly computed from the features of neighbouring test nodes.

To avoid this, every forward pass performed before the final test evaluation, including the training step itself, the periodic validation check used for early stopping, and the post-training threshold calibration, is restricted to the subgraph induced by non-test nodes. The complete graph, including edges incident to test-partition nodes, is used exactly once, for the final test-set forward pass reported in the next section. This preserves the temporal train/test boundary described in Section 2.1 throughout the entire pipeline, rather than only at the loss computation.

5 Experimental Results

The training run converged before reaching the maximum number of epochs: early stopping was triggered at epoch 115, with a best validation PR-AUC of 0.9714. The validation-optimal decision threshold was set to 0.9217, corresponding to a validation F1-score of 0.9279.

5.1 Test Set Performance

Class	Precision	Recall	F1-score	Support
Licit (0)	0.9709	0.9906	0.9807	15587
Illicit (1)	0.8086	0.5734	0.6710	1083

Overall accuracy on the test partition is 0.9635, with a ROC-AUC of 0.8986 and a PR-AUC of 0.6688. The corresponding confusion matrix is reported below.

	Predicted Licit	Predicted Illicit
Actual Licit	15440 (TN)	147 (FP)
Actual Illicit	462 (FN)	621 (TP)

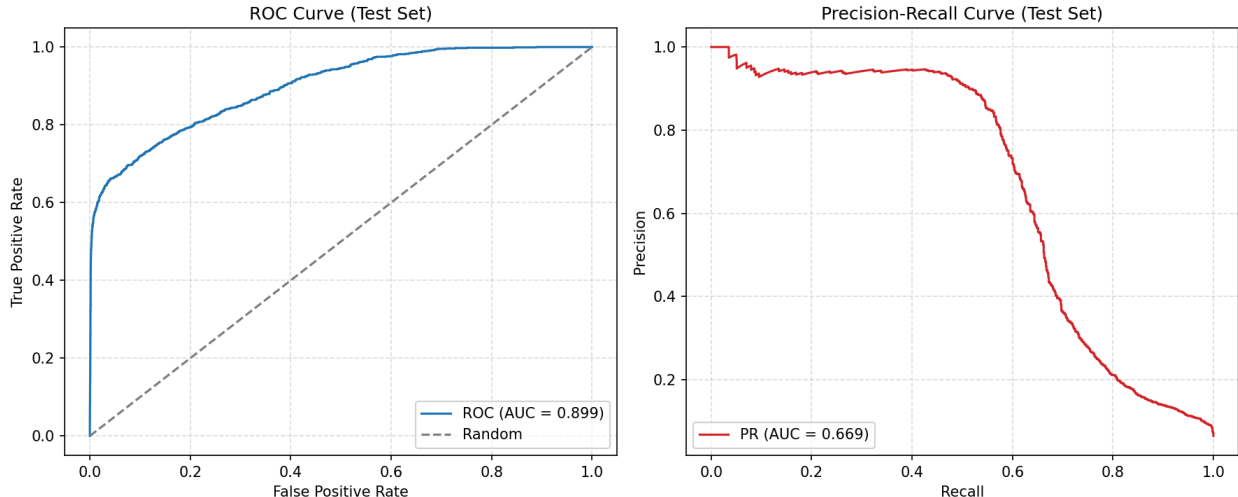


Figure 2: ROC curve (left) and Precision-Recall curve (right) on the test set.

The gap between the licit and illicit classes’ F1-scores follows directly from the imbalance discussed in Section 2.1: even with a weighted loss, illicit transactions remain intrinsically harder to separate from licit ones on this dataset, consistent with the observations of Weber et al. [3]. Precision (0.81) is noticeably higher than recall (0.57) on the illicit class, indicating a conservative decision boundary: approximately 43% of illicit transactions are missed, while few licit transactions are flagged as illicit (147 false positives out of 15,587). This follows directly from calibrating the threshold to maximise F1-score rather than recall. In a real AML pipeline, where missing an illicit transaction is typically costlier than a false alarm that is subsequently reviewed and dismissed, the threshold could instead be selected further along the precision-recall curve to trade precision for higher recall. In this implementation however, we didn’t proceed with that approach as we had no supporting models to filter out the false positives.

5.2 GPU Speedup

The following table reports the wall-clock cost of a full training step (forward pass, backward pass and optimizer step) and of an inference-only forward pass, on the Tesla T4 GPU versus the 2-core Intel Xeon CPU specified in Section 1.2.

Measurements follow the methodology described in the accompanying notebook:

- warmup iterations are discarded
- every timed region is bracketed by `torch.cuda.synchronize()`
- each figure is computed over 15 training or 30 inference trials.

Operation	Device	Mean $\pm$ Std (ms)	Min (ms)
Training step	GPU	84.22 $\pm$ 0.69	82.78
Training step	CPU	5000.52 $\pm$ 363.66	4667.18
Inference	GPU	46.66 $\pm$ 0.54	45.40
Inference	CPU	2075.90 $\pm$ 178.08	1923.11

Using the minimum measured time as reference, the GPU delivers a 56.4x speedup on the training step and a 42.4x speedup on inference over CPU execution, on the full graph.

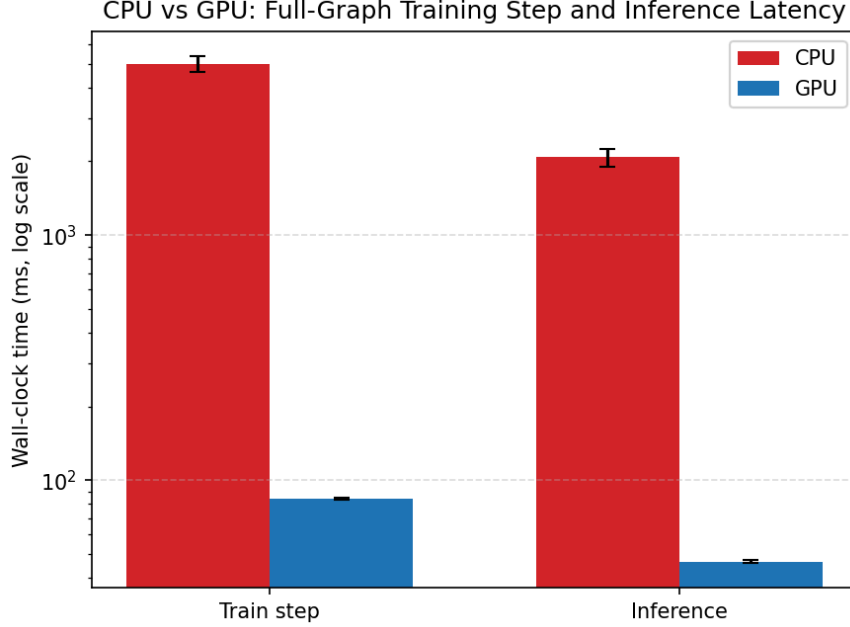


Figure 3: Mean wall-clock time (log scale) for a full training step and an inference pass, GPU versus CPU. Error bars show one standard deviation.

Two observations follow from the measurements:

- GPU timings are far more stable than CPU timings; relative standard deviation stays below 1.2% on GPU against 7-9% on CPU, consistent with dedicated compute units executing without the scheduling and cache contention a general-purpose CPU is subject to. The CPU runtime is also limited to 2 logical cores, which caps how much intra-op parallelism PyTorch can extract from the sparse aggregation kernels that dominate GraphSAGE’s cost, further widening the gap.
- The speedup is larger for the training step than for inference. The training step performs strictly more work per call (forward pass, backward pass and optimizer step, against a forward pass alone for inference), so the fixed per-call overhead of kernel launches and Python-level dispatch is amortised over a larger workload, letting the GPU’s parallelism advantage dominate more.

Peak GPU memory allocated during the benchmark was 1564 MB, well within the 15.6 GB available on the Tesla T4. Memory footprint is therefore not a limiting factor for scaling this architecture to the full Elliptic graph.

6 Conclusion

This work presented a skip-connected two-layer GraphSAGE encoder, combined with a weighted binary cross-entropy loss, PR-AUC-driven early stopping, and post-hoc threshold calibration, to detect illicit transactions in the Bitcoin Elliptic graph under a realistic temporal train/test split. The model achieves a ROC-AUC of 0.8986 and an illicit-class F1-score of 0.6710 on transactions from unseen, later timesteps, confirming that combining raw per-transaction features with learned neighbourhood aggregates is an effective strategy for this task, even under heavy class imbalance.

On the hardware side, the GPU delivers a 56.4x speedup on the training step and a 42.4x speedup on inference over CPU execution on the full graph, while keeping peak memory usage well within the budget of a single T4 GPU.

Bibliography

- [1] T. N. Kipf and M. Welling, “Variational Graph Auto-Encoders.” [Online]. Available: <https://arxiv.org/abs/1611.07308>
- [2] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes.” [Online]. Available: <https://arxiv.org/abs/1312.6114>
- [3] M. Weber *et al.*, “Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics.” [Online]. Available: <https://arxiv.org/abs/1908.02591>
- [4] T. N. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks.” [Online]. Available: <https://arxiv.org/abs/1609.02907>
- [5] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive Representation Learning on Large Graphs.” [Online]. Available: <https://arxiv.org/abs/1706.02216>
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [7] M. Fey and J. E. Lenssen, “Fast Graph Representation Learning with PyTorch Geometric.” [Online]. Available: <https://arxiv.org/abs/1903.02428>
- [8] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization.” [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [9] J. Davis and M. Goadrich, “The relationship between Precision-Recall and ROC curves,” in *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 233–240.